FarOS 操作系统开发手册

> FarOS (Fitness & Health microkernel Real-time Operating System)
>
> 版本: v1.0 | 日期: 2024-10

目录

1. 系统概述
2. 开发环境搭建
3. 内核编程指南
4. 驱动开发指南
5. 蓝牙开发指南
6. 运动健康功能开发
7. 附录

1. 系统概述

1.1 项目背景

FarOS 是面向运动健康垂直领域的专用型微内核操作系统，依托国家重点研发计划"主动健康和人口老龄化科技应对"重点专项（项目编号：2023YFC3606100）立项研发。

系统基于开源微内核基础架构完成场景化迭代与定制优化，聚焦运动健康智能设备专属应用场景，重点攻克多品类、多厂商智能健康设备跨域互操作核心难题。

1.2 系统架构

```
+-----------------------------------------+
|  Application Layer                      |
|  (brandy_standard, brandy_recovery, wstp)|
+-----------------------------------------+
|  Middleware Services                    |
|  + Health Service (运动健康服务)          |
|    - far_sensor    传感器抽象层          |
|    - far_activity  运动识别              |
|    - far_physio    生理指标              |
|    - far_sleep     睡眠分析              |
|    - far_health_data 数据管理            |
|  + Bluetooth Service (蓝牙服务)          |
|  + DFX (诊断与调试)                      |
|  + PM (电源管理)                         |
|  + FS (文件系统)                         |
|  + NV (非易失存储)                       |
+-----------------------------------------+
|  Kernel Layer                           |
|  + OSAL (操作系统抽象层)                 |
|  + LiteOS / FreeRTOS                    |
|  + DPAL (设备平台抽象层)                 |
```

```
+----------------------------------------+
|  Driver Layer                          |
|  + HAL (硬件抽象层)                     |
|  + Device Drivers                      |
+----------------------------------------+
|  Hardware                              |
|  + Brandy SoC (RISC-V 32-bit)          |
+----------------------------------------+
```

1.3 核心技术特性

特性	描述
超低功耗运行	精简系统冗余开销，优化算法与数据传输逻辑，延长设备续航
高安全高可靠	微内核隔离优势，完善权限管控、异常容错与风险预警机制
广谱设备兼容	适配感知类、电机健身类多品类终端，支持多品牌多型号设备快速接入
轻量化易部署	系统资源占用低，适配中低端硬件，提供标准化轻量化交钥匙解决方案

1.4 应用场景

- **大众主动健康监测**
- **智能化科学健身**
- **运动健康数据管控**
- **智慧体育场景建设**

---

2. 开发环境搭建

2.1 硬件环境

项目	要求
目标芯片	Brandy SoC (RISC-V RV32IMAC)
开发板	Brandy EVB
调试器	J-Link / J-Link OB
串口	USB转TTL (115200bps, 8N1)

2.2 软件环境

项目	版本
操作系统	Windows 10/11 或 Linux Ubuntu 20.04+
编译工具链	cc_riscv32_musl_fp_win (Linx LLVM)
构建系统	CMake 3.16+

项目	版本
Python	3.8+
Git	2.30+

2.3 编译与烧录

```
# 1. 配置编译目标
python build/script/build.py -t brandy-mcu-bt -c

# 2. 编译
python build/script/build.py -t brandy-mcu-bt -b

# 3. 烧录（通过J-Link）
python build/script/build.py -t brandy-mcu-bt -f

# 4. 查看串口日志
python build/script/build.py -t brandy-mcu-bt -l
```

---

3. 内核编程指南

3.1 任务管理

```
#include "cmsis_os2.h"

/* 创建任务 */
osThreadId_t task_id = osThreadNew(my_task_func, NULL, NULL);

/* 任务函数 */
static void my_task_func(void *arg)
{
    while (1) {
        /* 任务逻辑 */
        osDelay(100);  /* 延时100ms */
    }
}
```

3.2 消息队列

```
#include "soc_osal.h"

/* 创建队列 */
osal_msg_queue_create("my_queue", 10, &queue_id, 0, sizeof(my_msg_t));

/* 发送消息 */
osal_msg_queue_write_copy(queue_id, &msg, sizeof(msg), 0);

/* 接收消息 */
osal_msg_queue_read_copy(queue_id, &msg, &size, OSAL_MSGQ_WAIT_FOREVER);
```

3.3 定时器

```
#include "tcxo.h"

/* 延时 */
uapi_tcxo_delay_ms(100);   /* 延时100ms */
uapi_tcxo_delay_us(100);   /* 延时100us */

/* 获取系统时间 */
uint64_t ms = uapi_tcxo_get_ms();
uint64_t us = uapi_tcxo_get_us();
```

3.4 内存管理

```
/* 动态内存分配 */
void *ptr = malloc(size);
free(ptr);

/* 安全内存操作 */
memset_s(dst, dst_size, 0, dst_size);
memcpy_s(dst, dst_size, src, src_size);
```

---

4. 驱动开发指南

4.1 I2C驱动

```
#include "i2c.h"

/* 初始化I2C主机 */
i2c_data_t data = {
    .send_buf = tx_buf,
    .send_len = tx_len,
    .receive_buf = rx_buf,
    .receive_len = rx_len,
};

errcode_t ret = uapi_i2c_master_write(I2C_BUS_0, 0x50, &data);
```

4.2 GPIO驱动

```
#include "gpio.h"

/* 配置GPIO */
uapi_gpio_set_dir(GPIO_0, GPIO_DIRECTION_OUTPUT);
uapi_gpio_set_output_val(GPIO_0, GPIO_LEVEL_HIGH);

/* 读取GPIO */
gpio_level_t level;
uapi_gpio_get_input_val(GPIO_0, &level);
```

4.3 ADC驱动

```
#include "adc.h"

/* 配置ADC */
adc_scan_config_t config = {
    .type = ADC scanning_mode,
    .channel_count = 1,
    .channel_id = { ADC_CHANNEL_0 },
};

uapi_adc_auto_scan_config(&config);
```

---

5. 蓝牙开发指南

5.1 BLE GATT服务

FarOS 内置 BTS (Bluetooth Stack) 协议栈，支持 BLE GATT Server/Client 开发。

5.2 标准健康Profile

Profile	用途	状态
HRS (Heart Rate Service)	心率数据广播	待实现
RSCS (Running Speed and Cadence)	跑步速度和步频	待实现
BAS (Battery Service)	电池电量	已实现
DIS (Device Information)	设备信息	已实现

---

6. 运动健康功能开发

> **本章为手册核心章节**，详细介绍 FarOS 运动健康服务模块的架构设计、接口规范和调用方式。参考 Firstbeat 核心概念框架设计。

6.1 模块架构

运动健康服务模块采用**分层架构**设计，各层职责清晰、接口解耦：

```
+--------------------------------------------------+
|  Application / AT Command Layer                  |
|  - 健康数据展示、运动记录管理、调试命令            |
+--------------------------------------------------+
|  Health Service API Layer                        |
|  - far_health_common    公共类型与全局配置        |
|  - far_activity         运动识别（计步/活动类型）  |
|  - far_physio           生理指标（HR/HRV/卡路里等） |
|  - far_sleep            睡眠分析                  |
|  - far_health_data      数据管理                  |
+--------------------------------------------------+
```

```
|  Sensor Abstraction Layer                          |
|  - far_sensor          IMU/PPG传感器抽象            |
+----------------------------------------------------+
|  Driver Layer (I2C/SPI/GPIO/ADC)                   |
+----------------------------------------------------+
```

6.1.1 模块依赖关系

模块	依赖模块	说明
`far_sensor`	驱动层	直接访问硬件传感器
`far_activity`	`far_sensor`	使用IMU数据识别活动
`far_physio`	`far_sensor`	使用PPG数据计算生理指标
`far_sleep`	`far_sensor`, `far_physio`	使用IMU+HR数据分析睡眠
`far_health_data`	文件系统/NV	存储和查询健康记录

6.1.2 编译控制

运动健康服务通过 `SUPPORT_HEALTH_SERVICE` 宏控制编译：

```
/* 在 build/config/target_config/brandy/config.py 中添加：
 * 'SUPPORT_HEALTH_SERVICE': 'YES'
 */
```

6.2 传感器数据采集

6.2.1 传感器抽象层（`far_sensor`）

传感器抽象层提供统一的 IMU（加速度计+陀螺仪）和 PPG（光电容积脉搏波）传感器访问接口，屏蔽底层硬件差异。

核心数据结构：

```
/* IMU数据采样 */
typedef struct {
    int16_t  ax, ay, az;        /* 加速度，单位：mg */
    int16_t  gx, gy, gz;        /* 角速度，单位：mdps */
    uint32_t timestamp;         /* 时间戳，单位：ms */
} far_sensor_imu_data_t;

/* PPG原始数据采样 */
typedef struct {
    int32_t  green_raw;         /* 绿光LED原始ADC值 */
    int32_t  ir_raw;            /* 红外LED原始ADC值 */
    int32_t  red_raw;           /* 红光LED原始ADC值 */
    uint32_t timestamp;         /* 时间戳，单位：ms */
} far_sensor_ppg_data_t;
```

API接口：

函数	功能	返回值
`uapi_sensor_imu_init(config)`	初始化IMU传感器	`errcode_t`
`uapi_sensor_imu_read(data, max, &read)`	读取IMU数据	`errcode_t`
`uapi_sensor_imu_register_callback(cb, user)`	注册IMU数据回调	`errcode_t`
`uapi_sensor_ppg_init(config)`	初始化PPG传感器	`errcode_t`
`uapi_sensor_ppg_read(data, max, &read)`	读取PPG数据	`errcode_t`
`uapi_sensor_ppg_register_callback(cb, user)`	注册PPG数据回调	`errcode_t`
`uapi_sensor_set_sample_rate(type, rate)`	动态设置采样率	`errcode_t`
`uapi_sensor_enable(type, enable)`	使能/禁用传感器	`errcode_t`
`uapi_sensor_get_power_estimate(type, &power)`	获取功耗估算	`errcode_t`

调用示例：

```
#include "far_sensor.h"

/* 1. 初始化IMU传感器 */
far_sensor_imu_config_t imu_cfg = {
    .accel_rate = FAR_SENSOR_RATE_50_HZ,
    .gyro_rate = FAR_SENSOR_RATE_50_HZ,
    .mode = FAR_SENSOR_MODE_NORMAL,
    .accel_range_g = 8,
    .gyro_range_dps = 2000,
};
uapi_sensor_imu_init(&imu_cfg);

/* 2. 初始化PPG传感器 */
far_sensor_ppg_config_t ppg_cfg = {
    .sample_rate = FAR_SENSOR_RATE_25_HZ,
    .mode = FAR_SENSOR_MODE_NORMAL,
    .led_current_ma = 25,
    .enable_green = true,
    .enable_ir = false,
    .enable_red = false,
};
uapi_sensor_ppg_init(&ppg_cfg);

/* 3. 使能传感器 */
uapi_sensor_enable(0, true);  /* IMU */
uapi_sensor_enable(1, true);  /* PPG */

/* 4. 读取数据 */
far_sensor_imu_data_t imu_data[16];
uint32_t read_len = 0;
uapi_sensor_imu_read(imu_data, 16, &read_len);

/* 5. 异步数据流（推荐） */
void my_imu_callback(const void *data, uint32_t len, void *user_data)
```

```
{
    const far_sensor_imu_data_t *imu = (const far_sensor_imu_data_t *)data;
    /* 处理IMU数据... */
}
uapi_sensor_imu_register_callback(my_imu_callback, NULL);
```

6.2.2 传感器配置建议

应用场景	IMU采样率	PPG采样率	功耗估算
日常佩戴（计步）	50Hz	25Hz	~2.7mA
运动模式（跑步）	100Hz	50Hz	~5.5mA
睡眠监测	12.5Hz	12.5Hz	~1.1mA
高精度模式	200Hz	100Hz	~10mA

6.3 运动识别算法接口

6.3.1 功能概述

运动识别模块基于 IMU 传感器数据，实现：

- **计步**（Pedometer）：检测步态周期，统计步数
- **活动类型识别**（Activity Recognition）：识别静止/步行/跑步/骑行等活动类型
- **步频计算**（Cadence）：计算每分钟步数（SPM）
- **距离追踪**（Distance）：基于步数和步长估算行走距离

算法原理说明：

```
计步算法（峰值检测法）：
  1. 合成加速度幅值: acc_mag = sqrt(ax² + ay² + az²)
  2. 低通滤波（截止频率~3Hz）
  3. 检测局部峰值，峰值间隔在 300ms-1500ms 之间
  4. 峰值幅度超过自适应动态阈值

活动类型识别：
  1. 每3秒提取特征（均值、方差、主频率）
  2. 基于规则的决策树分类
     - REST:   acc_var < 1000 mg²
     - WALK:   1Hz < peak_freq < 2.2Hz, acc_var < 5000
     - RUN:    2Hz < peak_freq < 3.5Hz, acc_var > 5000
     - CYCLE:  gyro_var > threshold, acc_var < 3000

步长估算：
  step_length_cm = height_cm × (0.415 + 0.002 × (cadence_spm - 100))
```

API接口：

函数	功能	返回值
`uapi_activity_init(config)`	初始化运动识别	`errcode_t`
`uapi_activity_enable()`	使能运动识别	`errcode_t`
`uapi_activity_disable()`	禁用运动识别	`errcode_t`
`uapi_activity_get_step_count(&info)`	获取步数信息	`errcode_t`
`uapi_activity_reset_step_count()`	重置步数	`errcode_t`
`uapi_activity_get_type(&result)`	获取活动类型	`errcode_t`
`uapi_activity_get_cadence(&info)`	获取步频	`errcode_t`
`uapi_activity_get_distance(&info)`	获取距离	`errcode_t`
`uapi_activity_register_callback(cb, user)`	注册活动变化回调	`errcode_t`
`uapi_activity_get_daily_summary(&steps, &dist, &min)`	获取每日汇总	`errcode_t`

调用示例：

```
#include "far_activity.h"

/* 初始化运动识别 */
far_activity_config_t cfg = {
    .enable_step_count = true,
    .enable_activity_type = true,
    .enable_cadence = true,
    .enable_distance = true,
    .step_threshold = 150,
    .confidence_threshold = 70,
};
uapi_activity_init(&cfg);
uapi_activity_enable();

/* 注册活动变化回调 */
void on_activity_change(far_activity_type_t old_type,
                        far_activity_type_t new_type,
                        uint8_t confidence, void *user_data)
{
    const char *names[] = {"UNKNOWN","REST","WALK","RUN","CYCLE"};
    PRINT("Activity changed: %s -> %s (confidence=%d%%)\r\n",
          names[old_type], names[new_type], confidence);
}
uapi_activity_register_callback(on_activity_change, NULL);

/* 查询步数 */
far_step_info_t steps;
uapi_activity_get_step_count(&steps);
PRINT("Steps today: %lu\r\n", steps.steps_today);
```

```
/* 查询距离 */
far_distance_info_t dist;
uapi_activity_get_distance(&dist);
PRINT("Distance today: %lu m\r\n", dist.distance_today_m);
```

6.4 生理指标算法接口（参考Firstbeat）

6.4.1 功能概述

生理指标模块参考 **Firstbeat** 核心概念框架，提供以下高级生理参数：

指标	Firstbeat概念	算法基础	应用场景
心率监测	Heart Rate	PPG信号峰值检测	实时心率显示
HRV分析	HRV Analysis	RR间期时域/频域分析	自主神经评估
卡路里	Calorie Consumption	METs代谢当量模型	能量消耗统计
VO2max	VO2max Estimation	心率-速度回归模型	有氧能力评估
训练负荷	Training Load (EPOC)	运动后过量氧耗累积	训练强度评估
恢复时间	Recovery Time	EPOC衰减模型	训练恢复建议
压力指数	Stress Monitoring	日间HRV压力分析	压力管理
身体电量	Body Battery	HRV恢复+活动综合模型	能量管理

6.4.2 Firstbeat核心概念参考

EPOC（运动后过量氧耗）模型：

EPOC 是 Firstbeat 训练负荷计算的核心概念。运动后，身体需要消耗额外氧气来恢复到静息状态，这个额外的氧气消耗量即为 EPOC。

```
EPOC(t) = EPOC(t-1) + (MET(t) - 1) × k - EPOC(t-1) / τ

其中：
  MET(t) = 当前代谢当量
  k      = 个体校准系数
  τ      = EPOC衰减时间常数（30-60分钟）

训练负荷 = 单次运动中的EPOC峰值或累积量
```

METs（代谢当量）模型：

```
总卡路里 = BMR + 主动运动卡路里
BMR (Harris-Benedict):
  男: 88.362 + 13.397×体重(kg) + 4.799×身高(cm) - 5.677×年龄
  女: 447.593 + 9.247×体重(kg) + 3.098×身高(cm) - 4.330×年龄
主动运动卡路里 = MET × 体重(kg) × 时间(小时)
MET估算基于活动类型和心率区间
```

Body Battery（身体电量）：

```
BodyBattery(t) = BodyBattery(t-1) + Δcharge - Δdrain

充电（休息/睡眠）：
  Δcharge ∝ HRV恢复程度（HF增加）

放电（活动/压力）：
  Δdrain ∝ EPOC累积 + HR升高幅度

范围：0-100
```

6.4.3 API接口

心率监测：

函数	功能
`uapi_physio_hr_init()`	初始化心率监测
`uapi_physio_hr_get_current(&info)`	获取当前心率
`uapi_physio_hr_get_resting(&bpm)`	获取静息心率
`uapi_physio_hr_get_zone_distribution(&dist)`	获取心率区间分布

HRV分析：

函数	功能
`uapi_physio_hrv_analyze(&result)`	执行HRV分析（需≥5分钟数据）
`uapi_physio_hrv_feed_rr_data(rr, len)`	投喂RR间期数据

卡路里：

函数	功能
`uapi_physio_calorie_get_info(&info)`	获取卡路里信息
`uapi_physio_calorie_get_today(&total, &active)`	获取今日卡路里

VO2max：

函数	功能
`uapi_physio_vo2max_get(&result)`	获取VO2max估算值
`uapi_physio_vo2max_estimate(activity)`	启动VO2max估算（跑步/骑行）

训练负荷与恢复：

函数	功能
`uapi_physio_training_load_get(&load)`	获取训练负荷
`uapi_physio_training_load_reset()`	重置训练负荷
`uapi_physio_recovery_time_get(&recovery)`	获取恢复时间建议

压力与身体电量：

函数	功能
`uapi_physio_stress_get(&stress)`	获取压力指数
`uapi_physio_stress_enable(enable)`	使能/禁用压力监测
`uapi_physio_body_battery_get(&battery)`	获取身体电量
`uapi_physio_body_battery_enable(enable)`	使能/禁用身体电量

调用示例：

```
#include "far_physio.h"

/* 初始化 */
uapi_physio_init();
uapi_physio_hr_init();

/* 投喂心率数据（由PPG算法产生） */
uapi_physio_feed_hr_data(72, uapi_tcxo_get_ms());

/* 获取心率 */
far_hr_info_t hr;
uapi_physio_hr_get_current(&hr);
PRINT("HR: %u BPM, Zone: %d\r\n", hr.current_bpm, hr.current_zone);

/* HRV分析（需要至少5分钟连续RR数据） */
far_hrv_result_t hrv;
if (uapi_physio_hrv_analyze(&hrv) == ERRCODE_SUCC) {
    PRINT("HRV: SDNN=%ums, RMSSD=%ums, Score=%u\r\n",
          hrv.sdnn_ms, hrv.rmssd_ms, hrv.hrv_score);
}

/* 获取卡路里 */
far_calorie_info_t cal;
uapi_physio_calorie_get_info(&cal);
PRINT("Calories: total=%lu, active=%lu\r\n", cal.total_kcal, cal.active_kcal);

/* 获取压力指数 */
far_stress_info_t stress;
uapi_physio_stress_get(&stress);
PRINT("Stress: %u/100\r\n", stress.current_stress);

/* 获取身体电量 */
far_body_battery_t bb;
```

```
uapi_physio_body_battery_get(&bb);
PRINT("Body Battery: %u/100\r\n", bb.current_level);
```

6.5 睡眠分析接口

6.5.1 功能概述

睡眠分析模块基于 IMU 运动检测和 PPG 心率/HRV 数据，实现睡眠分期检测和睡眠质量评估。

睡眠分期算法：

```
清醒（Wake）：
  - 高频大幅度运动
  - 心率变异大

REM睡眠：
  - 几乎无运动（肌张力抑制）
  - 心率变异性高（类似清醒但身体不动）

浅睡（Light Sleep）：
  - 偶发微动
  - 心率逐渐降低
  - HRV适中

深睡（Deep Sleep）：
  - 几乎无运动
  - 心率最低且稳定
  - HRV低（副交感神经主导）
```

睡眠评分因素：

因素	权重	目标
总睡眠时长	40%	7-9小时
深睡比例	20%	15-25%
睡眠效率	20%	>85%
觉醒次数	10%	≤2次
HRV恢复	10%	睡眠期间HRV改善

API接口：

函数	功能
`uapi_sleep_init(config)`	初始化睡眠分析
`uapi_sleep_enable()`	使能睡眠分析
`uapi_sleep_start_tracking()`	手动开始睡眠追踪
`uapi_sleep_stop_tracking()`	手动停止睡眠追踪

函数	功能
`uapi_sleep_get_last_record(&record)`	获取最近睡眠记录
`uapi_sleep_get_score(&score)`	获取睡眠评分
`uapi_sleep_get_stages(&deep, &light, &rem, &awake)`	获取睡眠分期
`uapi_sleep_is_sleeping(&is_sleeping)`	检查是否在睡眠中

调用示例：

```
#include "far_sleep.h"

/* 初始化 */
far_sleep_config_t cfg = {
    .enable_auto_detect = true,
    .bedtime_hour = 255,        /* 自动检测 */
    .wake_time_hour = 255,      /* 自动检测 */
    .min_sleep_duration_min = 30,
    .enable_hrv_analysis = true,
};
uapi_sleep_init(&cfg);
uapi_sleep_enable();

/* 查询睡眠记录 */
far_sleep_record_t record;
if (uapi_sleep_get_last_record(&record) == ERRCODE_SUCC) {
    PRINT("Sleep: %umin total, score=%u\r\n",
          record.total_sleep_min, record.sleep_score);
    PRINT("  Deep: %umin, Light: %umin, REM: %umin, Awake: %umin\r\n",
          record.deep_sleep_min, record.light_sleep_min,
          record.rem_sleep_min, record.awake_min);
}

/* 查询睡眠评分 */
uint8_t score;
uapi_sleep_get_score(&score);
PRINT("Sleep Score: %u/100\r\n", score);
```

6.6 数据管理接口

6.6.1 功能概述

数据管理模块提供运动记录存储、历史数据查询、每日汇总和数据同步功能。支持 **NV存储** 和 **文件系统** 双模式。

存储架构：

存储模式	用途	数据类型
NV存储	快速访问的小数据	今日汇总、最近7天摘要
文件系统	大数据量长期存储	完整运动记录、睡眠分期

存储模式	用途	数据类型
双模式	冗余备份	关键数据同时存NV+文件系统

API接口：

函数	功能
`uapi_health_data_init(mode)`	初始化数据管理
`uapi_health_data_save_workout(&record)`	保存运动记录
`uapi_health_data_save_sleep(&record)`	保存睡眠记录
`uapi_health_data_get_workouts(&filter, &records, max, &count)`	查询运动记录
`uapi_health_data_get_daily_summary(&date, &summary)`	获取每日汇总
`uapi_health_data_delete_record(id)`	删除记录
`uapi_health_data_sync_start(callback, user)`	启动数据同步
`uapi_health_data_export(filename, &filter)`	导出记录

6.7 AT命令调试接口

运动健康服务提供 **20+** 个AT命令用于调试和测试：

AT命令	功能	示例输出
`AT^HEALTHINIT`	初始化服务	`Init OK`
`AT^HEALTHVER`	获取版本	`Version: 1.0.0`
`AT^HEALTHSTEP`	获取步数	`Steps: total=1234, today=5678`
`AT^HEALTHACT`	获取活动类型	`Activity: WALK, confidence=85%`
`AT^HEALTHHR`	获取心率	`HR: 72 BPM, Zone: AEROBIC`
`AT^HEALTHHRV`	HRV分析	`SDNN=45ms, RMSSD=35ms`
`AT^HEALTHCAL`	卡路里	`total=2150, active=450`
`AT^HEALTHVO2`	VO2max	`42 ml/kg/min, level=7`
`AT^HEALTHLOAD`	训练负荷	`EPOC=125, zone=OPTIMAL`
`AT^HEALTHSTR`	压力指数	`current=35/100`
`AT^HEALTHBAT`	身体电量	`current=78/100, charging`
`AT^HEALTHSLEEP`	睡眠记录	`total=480min, score=82`
`AT^HEALTHDAILY`	每日汇总	`steps=10000, distance=7500m`
`AT^HEALTHSENSOR`	传感器控制	`AT^HEALTHSENSOR=0,1`

AT命令	功能	示例输出
`AT^HEALTHSTATUS`	运行状态	`IMU=1.5mA, PPG=1.2mA`

6.8 性能指标与优化建议

6.8.1 算法性能指标

指标	目标值	测试方法
步数检测精度	≥ 95%	与标准计步器对比1000步
活动识别延迟	≤ 3s	从活动变化到识别出结果
心率监测精度	±3 BPM	与医用ECG对比
HRV分析精度	相关系数 ≥ 0.9	与医用Holter对比SDNN/RMSSD
卡路里计算误差	≤ 15%	参考间接测热法
VO2max估算误差	≤ 10%	与实验室测试结果对比
睡眠分期准确率	≥ 85%	与PSG多导睡眠监测对比

6.8.2 系统资源占用

指标	目标值	说明
算法任务内存	≤ 16KB	运行时堆栈+数据区
传感器采样功耗	≤ 1mA @ 50Hz	IMU持续采样
PPG采样功耗	≤ 3mA @ 25Hz	心率持续监测
算法处理延迟	≤ 10ms/周期	100Hz任务周期

6.8.3 功耗优化建议

1. **动态采样率调整**：根据活动类型自动调整传感器采样率
    - 静止时：IMU 12.5Hz，PPG 12.5Hz
    - 运动时：IMU 100Hz，PPG 50Hz
2. **批量数据处理**：传感器FIFO批量读取，减少CPU唤醒次数
3. **间歇性PPG采样**：非运动时采用间歇采样（每5秒采样1秒）
4. **深睡检测优化**：检测到深睡时降低所有传感器采样率
5. **事件驱动架构**：仅在数据就绪时唤醒算法任务

---

7. 附录

附录A: 运动健康API速查表

A.1 公共API

函数	文件
`uapi_health_service_init()`	`health_service_init.c`
`uapi_health_service_deinit()`	`health_service_init.c`
`uapi_health_get_version()`	`health_service_init.c`
`uapi_health_set_user_profile()`	`health_service_init.c`
`uapi_health_get_user_profile()`	`health_service_init.c`

A.2 传感器API

函数	文件
`uapi_sensor_imu_init()`	`far_sensor.c`
`uapi_sensor_imu_read()`	`far_sensor.c`
`uapi_sensor_ppg_init()`	`far_sensor.c`
`uapi_sensor_ppg_read()`	`far_sensor.c`
`uapi_sensor_set_sample_rate()`	`far_sensor.c`
`uapi_sensor_enable()`	`far_sensor.c`

A.3 运动识别API

函数	文件
`uapi_activity_init()`	`far_activity.c`
`uapi_activity_enable()`	`far_activity.c`
`uapi_activity_get_step_count()`	`far_activity.c`
`uapi_activity_get_type()`	`far_activity.c`
`uapi_activity_get_cadence()`	`far_activity.c`
`uapi_activity_get_distance()`	`far_activity.c`

A.4 生理指标API

函数	文件
`uapi_physio_init()`	`far_physio.c`
`uapi_physio_hr_get_current()`	`far_physio.c`
`uapi_physio_hrv_analyze()`	`far_physio.c`
`uapi_physio_calorie_get_info()`	`far_physio.c`
`uapi_physio_vo2max_get()`	`far_physio.c`
`uapi_physio_training_load_get()`	`far_physio.c`
`uapi_physio_recovery_time_get()`	`far_physio.c`
`uapi_physio_stress_get()`	`far_physio.c`
`uapi_physio_body_battery_get()`	`far_physio.c`

A.5 睡眠分析API

函数	文件
`uapi_sleep_init()`	`far_sleep.c`
`uapi_sleep_enable()`	`far_sleep.c`
`uapi_sleep_get_last_record()`	`far_sleep.c`
`uapi_sleep_get_score()`	`far_sleep.c`
`uapi_sleep_get_stages()`	`far_sleep.c`

A.6 数据管理API

函数	文件
`uapi_health_data_init()`	`far_health_data.c`
`uapi_health_data_save_workout()`	`far_health_data.c`
`uapi_health_data_get_daily_summary()`	`far_health_data.c`
`uapi_health_data_sync_start()`	`far_health_data.c`

附录B: 错误码表

错误码	值	说明
`ERRCODE_SUCC`	0x00000000	成功
`ERRCODE_FAIL`	0xFFFFFFFF	通用失败

错误码	值	说明
`ERRCODE_INVALID_PARAM`	0x80000001	参数无效
`ERRCODE_NOT_SUPPORT`	0x80000002	不支持的操作

附录C: 数据类型定义

C.1 活动类型枚举

```
typedef enum {
    FAR_ACTIVITY_TYPE_UNKNOWN   = 0,
    FAR_ACTIVITY_TYPE_REST      = 1,   /* 静止/久坐 */
    FAR_ACTIVITY_TYPE_WALK      = 2,   /* 步行 */
    FAR_ACTIVITY_TYPE_RUN       = 3,   /* 跑步 */
    FAR_ACTIVITY_TYPE_CYCLE     = 4,   /* 骑行 */
    FAR_ACTIVITY_TYPE_SWIM      = 5,   /* 游泳 */
    FAR_ACTIVITY_TYPE_HIKE      = 6,   /* 徒步 */
    FAR_ACTIVITY_TYPE_ELLIPTICAL = 7,  /* 椭圆机 */
    FAR_ACTIVITY_TYPE_ROWING    = 8,   /* 划船 */
    FAR_ACTIVITY_TYPE_STAIR     = 9,   /* 爬楼梯 */
    FAR_ACTIVITY_TYPE_SPORT_OTHER = 10,/* 其他运动 */
} far_activity_type_t;
```

C.2 睡眠分期枚举

```
typedef enum {
    FAR_SLEEP_STAGE_WAKE    = 0,   /* 清醒 */
    FAR_SLEEP_STAGE_REM     = 1,   /* REM快速眼动期 */
    FAR_SLEEP_STAGE_LIGHT   = 2,   /* 浅睡期 */
    FAR_SLEEP_STAGE_DEEP    = 3,   /* 深睡期 */
} far_sleep_stage_t;
```

C.3 心率区间枚举（Karvonen模型）

```
typedef enum {
    FAR_HR_ZONE_WARMUP      = 0,   /* 热身区 < 60% HRR */
    FAR_HR_ZONE_FAT_BURN    = 1,   /* 燃脂区 60-70% HRR */
    FAR_HR_ZONE_AEROBIC     = 2,   /* 有氧耐力区 70-80% HRR */
    FAR_HR_ZONE_THRESHOLD   = 3,   /* 乳酸阈值区 80-90% HRR */
    FAR_HR_ZONE_ANAEROBIC   = 4,   /* 无氧区 90-100% HRR */
} far_hr_zone_t;
```

C.4 用户档案结构体

```
typedef struct {
    uint8_t  gender;        /* 0=女, 1=男 */
    uint8_t  age;           /* 岁 */
    uint16_t height_cm;     /* 厘米 */
    uint16_t weight_kg;     /* 千克x10（0.1kg精度） */
    uint8_t  max_hr;        /* 自定义最大心率，0=自动计算 */
    uint8_t  rest_hr;       /* 实测静息心率 */
} far_user_profile_t;
```

附录D: 文件清单

D.1 头文件

文件路径	说明
`include/middleware/services/health/far_health_common.h`	公共类型定义
`include/middleware/services/health/far_sensor.h`	传感器抽象层
`include/middleware/services/health/far_activity.h`	运动识别
`include/middleware/services/health/far_physio.h`	生理指标
`include/middleware/services/health/far_sleep.h`	睡眠分析
`include/middleware/services/health/far_health_data.h`	数据管理

D.2 源文件

文件路径	说明
`middleware/services/health/health_service_init.c`	服务初始化与任务
`middleware/services/health/far_sensor.c`	传感器抽象实现
`middleware/services/health/far_activity.c`	运动识别实现
`middleware/services/health/far_physio.c`	生理指标实现
`middleware/services/health/far_sleep.c`	睡眠分析实现
`middleware/services/health/far_health_data.c`	数据管理实现
`middleware/services/health/CMakeLists.txt`	构建配置

D.3 AT命令扩展

文件路径	说明
`application/wstp/at_service/app/app_health_at_process.h`	AT命令头文件
`application/wstp/at_service/app/app_health_at_process.c`	AT命令实现

版本历史

版本	日期	修改内容
v1.0	2024-10	初始版本，添加运动健康服务模块完整接口规范和骨架代码

本文档为 FarOS 运动健康微内核操作系统开发手册，版权归 国体智慧体育技术创新中心 所有。